



Lab 2 Report

Double Buffering Using Multi-Threading

Name: Ahmad Bin Rashid

Roll No: 2023-CS-53

Instructor: Ma'am Aliya

Course: Compiler Construction Lab

University of Engineering and Technology

February 19, 2026

1 Introduction

Lexical analysis is the first phase of a compiler and involves extensive input/output (I/O) operations. Traditional single-buffer approaches often suffer from idle CPU time during I/O waits. To address this inefficiency, this lab implements a **double buffering mechanism using multi-threading**, allowing overlapping of file reading and character processing.

The objective of this lab is to design, implement, and evaluate a double buffer system and compare its performance against a single buffer baseline.

2 Objectives

- Implement a double buffering system for lexical analysis input.
- Apply multi-threading to overlap I/O and processing.
- Use synchronization techniques (semaphores and condition variables).
- Compare performance with a traditional single buffer approach.
- Demonstrate buffer transitions and correctness.

3 Project Structure

The project is organized into modular components as shown below:

```
lab2/  
> buffer_manager.py  
> threaded_reader.py  
> char_stream.py  
> single_buffer.py  
> test_generator.py  
> main.py  
> README.md
```

4 File Descriptions

4.1 buffer_manager.py

This module implements the core **BufferManager** class with the following features:

- Two buffers of configurable size (default 4096 bytes)
- Sentinel markers at the end of each buffer
- Thread-safe state tracking
- Collection of buffer usage statistics

4.2 `threaded_reader.py`

Implements the producer-consumer pattern using two synchronization methods:

- **ThreadedReaderSemaphore**: Uses semaphores
- **ThreadedReaderCondVar**: Uses condition variables

A factory function selects the appropriate implementation at runtime.

4.3 `char_stream.py`

Provides a clean interface for the lexical analyzer:

- `get_next_char()`
- `unget_char()`
- `get_lexeme()`
- `reset_lexeme_begin()`

4.4 `single_buffer.py`

Implements a traditional single buffer approach used as a baseline for performance comparison. It follows a sequential read-process-read pattern and records timing statistics.

4.5 `test_generator.py`

Generates realistic test files with configurable sizes (100KB, 500KB, 1MB, etc.) using varied source-code templates to simulate real compiler input.

4.6 `main.py`

Acts as the main driver program:

- Executes performance benchmarks
- Demonstrates buffer transitions
- Displays formatted output reports

5 Usage

5.1 Command Line Options

```
--input, -i          Input file path
--buffer-size, -b    Buffer size in bytes
--sync-method, -s    sem/semaphore or cv/condition
--runs, -r           Number of benchmark runs
--generate, -g       Generate test file
--no-demo            Skip buffer demo
--verbose, -v        Verbose output
```

6 Sample Output Overview

The program outputs detailed benchmark results including:

- File size and buffer configuration
- Number of buffer refills and switches
- Average I/O wait time
- Performance comparison between single and double buffering

7 Implementation Details

7.1 Synchronization Methods

7.1.1 Semaphores

The semaphore-based approach uses:

- `empty_sem`: Counts available empty buffers
- `full_sem`: Counts filled buffers

This follows the classical producer-consumer synchronization model.

7.1.2 Condition Variables

Condition variables provide a mutex-based flexible approach:

- `buffer_available`: Signals filled buffer
- `buffer_empty`: Signals processed buffer

7.2 Buffer States

Each buffer transitions through defined states:

`EMPTY → FILLING → FULL → PROCESSING`

7.3 Sentinel Character

A sentinel character (`\x00`) marks the end of valid data in each buffer, allowing efficient boundary detection without frequent bounds checking.

8 Experimental Results and Performance Evaluation

This section presents a detailed performance comparison between the single buffer and double buffer implementations using multi-threading. All experiments were conducted using semaphore-based synchronization.

8.1 Experiment 1: Medium File Size (102.45 KB)

8.1.1 Test Configuration

- Test File: `test_program.txt`
- File Size: 102.45 KB
- Buffer Size: 4096 bytes
- Synchronization Method: Semaphore
- Number of Runs: 3

8.1.2 Buffer Configuration

- Total File Size: 104905 bytes
- Buffer Size: 4096 bytes
- Synchronization Method: Semaphore

8.1.3 Reading Statistics

Table 1: Reading Statistics for 102.45 KB File

Metric	Single Buffer	Double Buffer
Buffer Refills / Switches	26	25
Read Operations	26	26
Average Refill / Fill Time	0.45 ms	0.42 ms
Total I/O Wait Time	11.70 ms	–

8.1.4 Performance Comparison

Table 2: Performance Comparison for 102.45 KB File

Mode	Processing Time (ms)
Single Buffer	85.23
Double Buffer	78.45

Performance Improvement: 7.9% (Double buffering is faster)

Characters Processed: 104905

8.1.5 Observation

For medium-sized input files, double buffering reduces I/O wait time by overlapping file reading with character processing, resulting in noticeable performance gains.

8.2 Experiment 2: Large File Size (1.05 MB)

8.2.1 Test Configuration

- Test File: `test_program.txt`
- File Size: 1.05 MB
- Buffer Size: 4096 bytes
- Synchronization Method: Semaphore
- Number of Runs: 5

8.2.2 Buffer Configuration

- Total File Size: 1,097,527 bytes
- Buffer Size: 4096 bytes
- Synchronization Method: Semaphore

8.2.3 Reading Statistics

Table 3: Reading Statistics for 1.05 MB File

Metric	Single Buffer	Double Buffer
Buffer Refills / Switches	257	256
Read Operations	257	257
Average Refill / Fill Time	41.99 μ s	64.19 μ s
Total I/O Wait Time	10.79 ms	–

8.2.4 Performance Comparison

Table 4: Performance Comparison for 1.05 MB File

Mode	Processing Time (seconds)
Single Buffer	2.16
Double Buffer	1.55

Performance Improvement: 28.0% (Double buffering is significantly faster)

Characters Processed: 1,049,060

8.2.5 Observation

As file size increases, the performance advantage of double buffering becomes more pronounced. The ability to overlap I/O and processing results in a substantial reduction in total execution time, demonstrating the scalability of the double buffering approach.

9 Performance Comparison

Experimental results show that double buffering consistently outperforms the single buffer approach. By overlapping I/O and processing, the total execution time is reduced.

- Single Buffer: Higher I/O wait time
- Double Buffer: Reduced wait time and faster processing

10 Edge Cases Handled

1. End-of-file (EOF) detection
2. Lexemes spanning across buffers
3. Small files and empty files
4. Graceful thread termination

11 Limitations

Due to Python's Global Interpreter Lock (GIL), true CPU parallelism is limited. However, since file I/O releases the GIL, performance improvements are still observed. For maximum efficiency, the same approach can be implemented in C or C++.

12 Conclusion

This lab successfully demonstrates the effectiveness of double buffering using multi-threading. The implementation improves I/O efficiency, reduces processing time, and provides a strong conceptual foundation for real-world compiler design.